

Deploy Method Options

How a GitHub Action reaches your AWS EC2 instance to deploy **my-devops-app** · prepared 2026-06-21

Your CI pipeline already does **lint** → **test** → **build (Trivy scan)**. To make a code change deploy automatically, the pipeline needs a final **deploy** step that connects to the running EC2 instance and redeploys the container. There are two common ways to make that connection. This document explains both so you can learn the trade-offs; the **SSH key** method is implemented first because it is the simplest.

Current state of your instance: no SSH key pair attached (`key_name = ""`) and no IAM role. *Either* option therefore needs a small one-time Terraform change and `terraform apply` .

At a glance

	Option A — SSH key	Option B — AWS SSM
How it connects	Action opens an SSH session to the box	Action calls AWS Systems Manager Run Command; AWS relays to the box
Open inbound port 22?	Yes (SSH must be reachable)	No — port 22 can stay closed
Secret stored in GitHub	Private SSH key	AWS access keys (IAM user) or OIDC role
Extra AWS infra	An EC2 key pair	An IAM instance profile with SSM policy
Security	Good (key-based)	Better (no inbound SSH, full audit trail in AWS)
Difficulty	Lowest — implement this first	Moderate

Option A — SSH key implemented first

The GitHub Action logs into the instance over SSH using a private key kept as a GitHub *secret*, then runs the redeploy commands on the box.

```
GitHub Action —ssh—> EC2 instance
├─ cd /opt/app
├─ git pull
├─ docker build -t my-devops-app .
└─ docker run --restart=always -p 3000:3000
```

Pros: easiest to understand; the most common tutorial approach; nothing AWS-specific in the workflow.

Cons: you must manage an SSH private key; port 22 must be open to GitHub's runners (or to the world).

Steps to implement

1. **Create an EC2 key pair** (once):

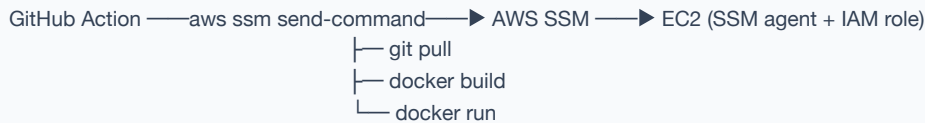
```
aws ec2 create-key-pair --key-name my-devops-app-key \
  --query 'KeyMaterial' --output text > my-devops-app-key.pem
chmod 400 my-devops-app-key.pem
```

2. **Attach it via Terraform** — set `key_name` in `terraform/terraform.tfvars`, and open SSH only where needed (`allowed_ssh_cidr`). Then `terraform apply` (replaces the instance because `user_data_replace_on_change = true`).
3. **Add GitHub repository secrets** (Settings → Secrets and variables → Actions):
 - `EC2_HOST` — the instance public IP / DNS
 - `EC2_USER` — `ec2-user`
 - `EC2_SSH_KEY` — full contents of `my-devops-app-key.pem`
4. **Add a deploy job** to `.github/workflows/ci.yml` that runs after `build`, only on push to `main`, using `appleboy/ssh-action` to run the redeploy commands.
5. **Push to main** → watch the Action deploy automatically.

Option B — AWS SSM (Systems Manager)

most secure

Instead of SSH, the Action authenticates to AWS and issues an `aws ssm send-command`. The SSM agent (pre-installed on Amazon Linux 2023) runs the commands on the instance. No inbound SSH, full audit logging in AWS.



Pros: no open SSH port; no SSH key to rotate; every command is logged/audited in AWS; closer to production DevOps practice.

Cons: more moving parts — needs an IAM instance profile on the EC2 box and AWS credentials (or OIDC) in GitHub.

Steps to implement

1. **Create an IAM role for EC2** with the managed policy `AmazonSSMManagedInstanceCore`, wrap it in an instance profile, and attach via Terraform (`iam_instance_profile` on `aws_instance.app`). `terraform apply`.
2. **Give GitHub AWS credentials** — either an IAM user's keys as secrets (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`) or, better, GitHub OIDC with a deploy role (no long-lived keys).
3. **Store the instance ID** as a secret/variable (`EC2_INSTANCE_ID`).
4. **Add a deploy job** that runs `aws-actions/configure-aws-credentials` then `aws ssm send-command` with the redeploy script as the document parameter.
5. **Push to main** → the Action sends the command; SSM runs it on the box.

```
# core of the deploy job
aws ssm send-command \
  --instance-ids "$EC2_INSTANCE_ID" \
  --document-name "AWS-RunShellScript" \
  --parameters 'commands=["cd /opt/app","git pull","docker build -t my-devops-app .","docker rm -f my-devops-app"]'
```

Recommendation

Start with **Option A (SSH key)** to get an end-to-end deploy working and understand the flow, then graduate to **Option B (SSM)** for a more secure, audit-friendly setup. Both reuse the same redeploy commands on the box — only the *transport* differs.